

Relatório PBL

MI - Concorrência e Conectividade

Problema 1

¹ Alisson Silva de Pinho, ¹ Davi Oliveira Santana Carvalho, ¹ Sinval Victor Agostinho Mota

¹Engenharia da Computação - Universidade Estadual de Feira de Santana (UEFS)
44036-900 - Feira de Santana - Bahia - Brasil

Resumo. *O crescimento da frota de veículos elétricos no Brasil tem impulsionado a necessidade por soluções eficazes de recarga. Entretanto, a infraestrutura ainda é limitada e carece de comunicação eficiente entre os veículos e os pontos de recarga, dificultando o uso prático desses automóveis. Para enfrentar esse desafio, este trabalho propõe um sistema cliente-servidor baseado em um protocolo próprio, que viabiliza a comunicação estruturada entre veículos, estações de recarga e o servidor central. A implementação foi realizada com o uso de sockets TCP e mensagens no formato JSON, garantindo clareza e consistência na troca de dados e flexibilidade na expansão do sistema. Os resultados demonstraram comunicação confiável, distribuição eficiente da carga entre os pontos e controle automatizado de reserva e liberação das estações, validando a efetividade da solução proposta.*

1. Introdução

O crescimento da frota de veículos elétricos no Brasil tem sido expressivo nos últimos anos, impulsionado por incentivos à sustentabilidade e avanços na tecnologia automotiva. Segundo a Associação Brasileira do Veículo Elétrico, o país superou a marca de 170 mil veículos eletrificados em circulação em 2024, refletindo um aumento contínuo na adoção dessa tecnologia [Associação Brasileira do Veículo Elétrico 2024]. Com isso, surge a necessidade de infraestrutura adequada para atender à demanda por recarga desses veículos.

Apesar do avanço no número de veículos, a infraestrutura nacional de recarga ainda é limitada e pouco integrada. A falta de comunicação em tempo real entre veículos e estações de recarga dificulta o uso inteligente dos recursos disponíveis, causando filas, desperdício de tempo e ineficiência no sistema como um todo.

Este trabalho propõe uma solução para esse cenário por meio do desenvolvimento de um sistema cliente-servidor baseado em um protocolo de comunicação próprio. O protocolo viabiliza a troca estruturada de informações entre veículos elétricos, estações de recarga e um servidor central, permitindo controle e monitoramento em tempo real.

A comunicação entre os componentes é realizada por meio da interface de sockets TCP, amplamente utilizada em redes de computadores por permitir conexões confiáveis e orientadas à conexão entre processos distribuídos [Tanenbaum and Wetherall 2011]. A solução foi implementada utilizando sockets TCP para garantir uma comunicação confiável, com mensagens estruturadas no formato JSON. Cada operação é identificada por um código numérico, facilitando o tratamento de mensagens, status e erros. O sistema foi testado em contêineres Docker, simulando diferentes instâncias de veículos e postos.

Os testes realizados demonstraram que a comunicação entre os componentes foi robusta e eficiente. A solução permitiu a automação da reserva e liberação de estações de recarga, além de promover a distribuição otimizada da carga entre os postos disponíveis. Esses resultados validam a efetividade do protocolo proposto frente aos desafios apresentados.

2. Fundamentação Teórica

Esta seção apresenta os conceitos e tecnologias utilizados no desenvolvimento da solução proposta. São abordados aspectos relacionados à comunicação em redes, modelagem de protocolos, troca estruturada de dados e virtualização de ambientes para testes.

2.1. Protocolos de Comunicação e a Pilha TCP/IP

A pilha de protocolos TCP/IP é um modelo de comunicação em redes que permite a troca de dados entre dispositivos de forma estruturada e padronizada. Ela é composta por camadas hierárquicas, sendo a camada de transporte responsável pela entrega confiável dos dados. O protocolo TCP (Transmission Control Protocol) oferece mecanismos como verificação de integridade, controle de fluxo e retransmissão em caso de perda, garantindo que as mensagens cheguem corretamente ao destino [Tanenbaum and Wetherall 2011]. Esses atributos justificam sua escolha neste projeto, que exige precisão na troca de dados entre veículos, estações e servidor.

2.2. Sockets e Comunicação Cliente-Servidor

A comunicação entre os módulos do sistema proposto foi implementada com base no modelo cliente-servidor, utilizando sockets TCP. Sockets são interfaces de software que permitem a comunicação bidirecional entre processos em uma rede. Essa abordagem viabiliza a troca direta de mensagens entre as entidades participantes do sistema, como carros e pontos de recarga, de maneira escalável e eficiente. A simplicidade da API de sockets e sua compatibilidade com múltiplas linguagens e plataformas tornam-na ideal para aplicações distribuídas.

2.3. Desenvolvimento de Protocolos Proprietários

Protocolos de comunicação proprietários são desenvolvidos quando há necessidade de controle sobre o formato e a semântica das mensagens trocadas. Neste projeto, foi implementado um protocolo customizado que define um conjunto de códigos numéricos para cada operação (como requisição de recarga, liberação de posto e atualização de status), junto com formatos de dados estruturados. O protocolo possibilita a padronização da comunicação e facilita o tratamento de mensagens e erros durante o funcionamento do sistema.

2.4. Mensagens Estruturadas com JSON

Para representar as mensagens entre os diferentes componentes do sistema, foi adotado o formato JSON (JavaScript Object Notation). JSON é uma linguagem de intercâmbio de dados leve, legível por humanos e de fácil manipulação por máquinas. Sua estrutura baseada em pares chave-valor torna a serialização e desserialização simples e rápida, sendo amplamente utilizada em aplicações de rede [Kurose and Ross 2013]. Essa escolha contribuiu para a clareza e manutenção do protocolo de comunicação criado.

2.5. Contêineres Docker e Ambientes de Simulação

A utilização de contêineres Docker permitiu a criação de um ambiente isolado e replicável para testes do sistema. Cada instância dos clientes (carros e postos) e do servidor foi executada em contêineres independentes, facilitando a simulação de cenários diversos com múltiplos nós em execução simultânea. Essa abordagem garantiu consistência nos testes, controle do ambiente e facilidade de implantação durante o desenvolvimento.

2.6. Sistemas Distribuídos e Sincronização de Recursos

Sistemas distribuídos são compostos por múltiplos nós que operam de forma coordenada para atingir um objetivo comum. Uma das principais dificuldades nesse tipo de sistema é o gerenciamento e a sincronização do acesso a recursos compartilhados, como os pontos de recarga neste projeto. O servidor central atua como mediador, controlando reservas e liberações com base em mensagens recebidas em tempo real, evitando conflitos e garantindo integridade nas operações [Tanenbaum and Wetherall 2011].

3. Metodologia

O desenvolvimento do sistema foi conduzido com base em uma abordagem incremental, estruturada nas seguintes etapas principais:

1. **Planejamento e organização do trabalho:** Foram realizadas reuniões semanais para entendimento do problema, divisão de tarefas e acompanhamento do progresso. Nesta fase, também foram definidos os requisitos do sistema, tanto funcionais quanto não funcionais.
2. **Escolha das tecnologias:** Após análise das demandas do projeto, optou-se pela linguagem Go devido à sua simplicidade, suporte a sockets e integração facilitada com contêineres Docker.
3. **Modelagem da comunicação:** Foi definida a arquitetura cliente-servidor e criada a estrutura de um protocolo próprio baseado em TCP. Cada operação do sistema foi associada a um código numérico e estrutura de mensagem em JSON, padronizando a troca de dados.
4. **Desenho do fluxo de mensagens:** Elaborou-se um fluxograma representando as requisições e respostas entre os componentes do sistema, contemplando ações como reserva de vaga, início/fim da recarga, envio de status e liberação do ponto.
5. **Implementação do sistema:** O sistema foi desenvolvido com uso de sockets TCP, considerando o protocolo definido. A lógica de comunicação, tratamento de mensagens e sincronização de recursos foi implementada conforme o projeto.
6. **Simulação com contêineres Docker:** Para testar o funcionamento da solução, foram utilizados contêineres Docker representando múltiplos veículos e estações de recarga. Isso permitiu verificar o comportamento do sistema em diferentes cenários simulados.
7. **Validação e análise dos resultados:** As interações entre os módulos foram monitoradas, e os dados coletados permitiram avaliar a eficiência da comunicação, o controle sobre os recursos e a consistência das respostas do servidor.

4. Resultados e Discussões

Nesta seção, são apresentados os resultados obtidos com a implementação do sistema, seguidos de discussões sobre seu funcionamento, limitações e possíveis melhorias.

4.1. Resultados Obtidos

O sistema desenvolvido consiste em um servidor central que gerencia a comunicação entre clientes do tipo carro e clientes do tipo estação de recarga, essa relação é visualizada na Figura 1. Durante os testes, o sistema demonstrou ser funcional, permitindo o uso das funcionalidades descritas nessa sessão.

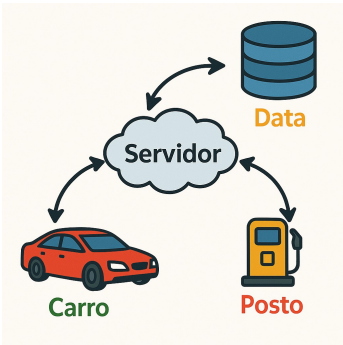


Figura 1. Diagrama da Arquitetura do Sistema

A Figura 2 apresenta as requisições e como as mensagens são transmitidas na arquitetura cliente-servidor

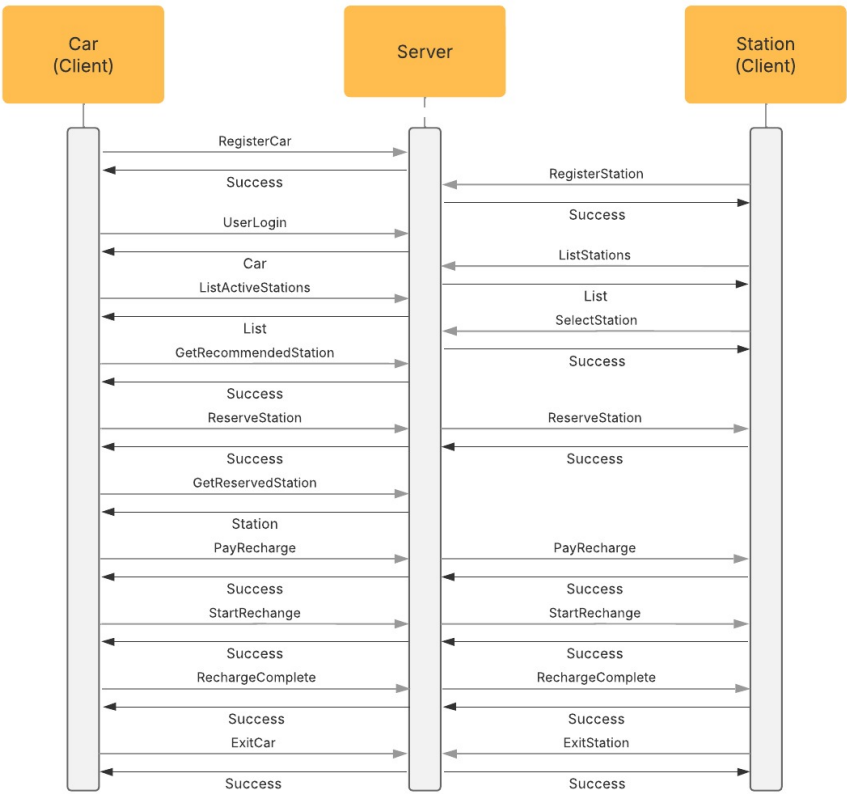


Figura 2. Diagrama Sequencial das Requisições

4.1.1. Registro de Carros e Estações

O sistema permite que carros e estações sejam registrados no servidor, com os dados sendo armazenados em arquivos JSON (`cars.json` e `stations.json`), o que garante persistência. Os testes demonstraram que o processo de registro funciona corretamente para múltiplos carros e estações, com os dados sendo atualizados de forma adequada.

4.1.2. Recomendação de Estações

O servidor é capaz de calcular a estação mais viável para um carro com base em dois critérios principais: a distância entre o carro e a estação, calculada utilizando a distância de Manhattan (a soma das diferenças absolutas entre as coordenadas dos dois pontos) e o tempo de espera estimado, baseado no número de carros na fila da estação. Esses dados são combinados em uma fórmula de score, onde a estação com o menor score é selecionada como a mais viável. Durante os testes, o sistema demonstrou eficiência ao recomendar estações que minimizam o tempo total de deslocamento e espera, garantindo uma experiência otimizada para o usuário.

4.1.3. Reserva de Estações

Carros podem reservar estações disponíveis enviando uma solicitação ao servidor, que verifica a disponibilidade da estação e atualiza os dados tanto do carro quanto da estação para refletir a reserva. O servidor adiciona o carro à fila da estação e incrementa o contador de carros aguardando. Além disso, o sistema foi projetado para impedir reservas duplicadas, garantindo que um carro não possa reservar a mesma estação mais de uma vez. Durante os testes, foi confirmado que apenas carros que estão na fila podem acessar a estação, assegurando que o fluxo de reservas e atendimentos seja realizado de forma ordenada e consistente.

4.1.4. Pagamento e Recarga

O sistema permite que carros realizem pagamentos utilizando Pix ou cartão de crédito antes de iniciar a recarga. O pagamento é validado pelo servidor, que verifica se o carro está na estação correta e se é o próximo na fila. Após a validação, o pagamento é registrado no histórico do carro. Somente após o pagamento, a recarga é então simulada com base no tempo necessário para carregar a bateria, calculado proporcionalmente ao nível atual de carga. Os testes confirmaram que o processo de pagamento e recarga funciona corretamente, garantindo que os dados sejam atualizados no servidor e que o fluxo de recarga seja concluído conforme o esperado.

4.1.5. Gerenciamento de Conexões

O sistema gerencia as conexões de carros e estações de forma eficiente. Durante o login, os carros e estações são registrados no servidor, que mantém um mapa de conexões

ativas. Isso permite que o servidor identifique cada cliente de forma única e encaminhe mensagens ou atualizações diretamente para o destinatário correto.

Quando um cliente desconecta, o servidor remove suas informações do mapa de conexões e atualiza os dados persistidos, como reservas pendentes ou filas de espera. No caso de estações, o servidor também remove a estação da lista de estações ativas, garantindo que a listagem exibida aos clientes permaneça consistente. Para carros, qualquer reserva ou recarga em andamento é finalizada, e o estado do carro é salvo no arquivo JSON para garantir a persistência dos dados. Esses mecanismos foram testados e demonstraram funcionar corretamente, mesmo em cenários de desconexões inesperadas.

4.2. Testes e Simulação de Trajeto dos Carros

O sistema implementa uma funcionalidade para simular o movimento de carros até uma estação de recarga, levando em conta coordenadas de origem e destino, velocidade do carro e nível de bateria. Durante a simulação, o carro percorre o trajeto gradualmente, atualizando suas coordenadas a cada passo e decrementando a bateria proporcionalmente à distância percorrida.

Durante a execução da simulação, o sistema fornece feedback em tempo real, exibindo no terminal as coordenadas atuais do carro e o nível de bateria restante. Ao alcançar a estação, a simulação é encerrada com uma mensagem indicando a chegada ao destino.

5. Conclusão

O desenvolvimento deste sistema de recarga de carros elétricos inteligente proporcionou uma experiência prática e desafiadora, permitindo a aplicação de conceitos fundamentais de programação, redes e conectividade. O projeto foi bem-sucedido na implementação de funcionalidades essenciais.

No entanto, algumas limitações foram identificadas, como a falta de tratamento de conexões e dependência de arquivos JSON para persistência de dados, o que pode comprometer a escalabilidade do sistema. A inclusão de um banco de dados relacional, bem como uma interface gráfica, seriam melhorias futuras.

Referências

- Associação Brasileira do Veículo Elétrico (2024). Balanço da eletromobilidade no Brasil em 2024. Acesso em: abril de 2025.
- Kurose, J. F. and Ross, K. W. (2013). *Redes de Computadores e a Internet: Uma Abordagem Top-Down*. Addison-Wesley, São Paulo, 5 edition.
- Tanenbaum, A. S. and Wetherall, D. J. (2011). *Redes de Computadores*. Pearson Prentice Hall, São Paulo, 5 edition.